PONTIFICIA UNIVERSIDAD CATÓLICA DE CHILE
Facultad de Ingeniería
IIC2613 Inteligencia Artificial
2026-1

Tarea 2
Aprendizaje Automático

Fecha de entrega: Viernes 10 de abril a las 23:59 hrs

Aspectos generales

Formato y plazo de entrega

El formato de entrega es **únicamente** un archivo con extensión `.ipynb` para tanto las respuestas de código como teóricas. El lugar de entrega es en el repositorio de la tarea, en la branch por defecto, hasta el viernes 10 de abril a las 23:59 hrs. Para crear tu repositorio, debes entrar en el enlace del anuncio de la tarea en Canvas. Por último, recuerda que los cupones de atraso son días **reales** (hábiles o no) extra.

Integridad Académica

Este curso se adhiere al Código de Honor establecido por la universidad, el cual tienes el deber de conocer como estudiante. Todo el trabajo hecho en esta tarea debe ser **totalmente individual**. La idea es que te des el tiempo de aprender estos conceptos fundamentales, tanto para el curso, como para tu formación profesional. Las dudas se deben hacer exclusivamente al cuerpo docente a través de las *issues* en GitHub.

Por otra parte, sabemos que estás utilizando material hecho por otras personas, por lo que es importante reconocerlo de la forma apropiada. Todo lo que obtengas de internet debes citarlo de forma correcta (ya sea en formato APA, ICONTEC o IEEE). Cualquier falta a la ética y/o a la integridad académica será sancionada con la reprobación del curso y los antecedentes serán entregados a la Dirección de Pregrado.

El uso de agentes inteligentes—como ChatGPT, Gemini, Copilot u otros—está permitido en las tareas siempre y cuando se indique, al inicio de la respuesta, que si se utilizaron, que se especifique de manera clara su uso (al menos el o los prompts y respuestas) y que las respuestas de estos sean modificadas sustancialmente.

Comentarios adicionales

El objetivo de esta tarea es que puedan utilizar distintos algoritmos para resolver problemas de clasificación y regresión, aplicándolos en problemas donde pueden ser de gran utilidad. Es fundamental que pongan énfasis en las justificaciones de sus respuestas, cuidando la redacción, ortografía; manteniendo el código ordenado y comentado. Aquellas respuestas que solo presenten resultados o código (sin contexto ni comentarios) no serán consideradas, mientras que tareas desordenadas pueden ser objeto de descuentos.

Recursos para realizar la tarea

Pandas

Esta librería es especialmente útil para manejar conjuntos de datos en forma de tablas. Es una extensión de **NumPy**, por lo que los cálculos matemáticos realizados usando las clases de esta librería son muy eficientes, aunque sus mayores atributos están en la forma en la que se indexan los datos. La clase **DataFrame** permite hacer un manejo muy eficiente de estos.

En general, se denominará **DataFrame** a la tabla de datos cuando estos ya estén cargados, y **Series** a las columnas. Para importar la librería, por convención, utilizamos el diminutivo **pd**:

```
import pandas as pd
```

Por convención, un **DataFrame** se denomina **df**. Si queremos tener una idea de lo que contiene, podemos usar el método **head**, que despliega los primeros cinco elementos (filas) del dataset, lo que es bastante rápido y ordenado.

Por otro lado, podemos conocer el número de filas y columnas de un **DataFrame** con el atributo **shape**. También, pueden resultar útiles los métodos **info** y **describe**. El primero devuelve características (número de columnas, cuáles son y número de elementos no nulos), y el segundo proporciona información básica estadística del conjunto de datos.

Por otro lado, si queremos limpiar el **DataFrame** de datos indeseados, deberemos conocerlos primero. Para ello, se puede utilizar el método **isna**. Los elementos nulos aparecen con el denominativo **NaN**, y el método **isna** devolverá un booleano con un valor correspondiente a si se encuentra dicho denominativo o no. De la misma manera, el método **duplicated** nos ayudará a encontrar duplicados.

Navegar por los datos es relativamente intuitivo. Una columna o serie puede ser accedida como si fuera un atributo de valor **name**, por ejemplo, **df.name**, o como si fuera un diccionario con una llave: **df["name"]**. Para acceder a dos series se utiliza la extensión **df[["name1", "name2"]]**. Una lista de todas las series disponibles se obtiene con **df.columns**. Finalmente, para acceder a un elemento dentro de una columna se debe utilizar su índice luego de haber llamado a la serie. Por ejemplo, al elemento **354** de **name** se accede haciendo **df["name"][354]**.

Una vez hemos podido acceder a ciertas series, o a elementos de ellas, es posible trabajar de manera numérica. Algunas operaciones matemáticas son bastante simples y directas, mientras que otras pueden requerir un poco más de investigación en la documentación de la librería. Por ejemplo:

```
# Esta es una operacion simple sobre dos celdas del DataFrame
q = df["name1"][354] + df["name2"][726]

# Estas son operaciones sobre toda la serie
m = df["name3"].mean()
s = df["name4"].sum()
```

Una operación numérica interesante sobre la serie es la de **value_counts**, que retornará la cantidad de elementos de cada clase de una serie. En particular, las clases de esa serie se pueden recuperar con el método **unique**.

Existen accesos más avanzados que se pueden invocar con el método **loc** o el método **groupby**. Estos métodos permiten trabajar con grandes cantidades de datos, hacer consultas y agrupaciones. El método **loc** permite filtrar filas o columnas mediante una etiqueta o un booleano. Un ejemplo puede ser:

```
df.loc[df["name1"] > 6, ["name2"]]
```

En este ejemplo se filtrarán todas las filas cuya serie de nombre **name1** tenga un valor **mayor que 6**, pero se recuperarán los valores de la serie **name2** de esas filas.

Por otro lado, el método **groupby** retornará un objeto con varias propiedades interesantes. Típicamente se utiliza sobre valores no numéricos a los cuales se les pueden aplicar otras operaciones o métodos:

```
df.groupby(["name1"]).sum()
```

Si la serie **name1** tiene **5 instancias**, el método mostrará una tabla de **5 filas** para las cuales intentará sumar sus valores en las otras series.

Finalmente, para la manipulación de archivos, existen dos tipos de métodos principales. Si lo que se desea es guardar un DataFrame procesado en un archivo `.csv`, se puede usar el método **to_csv**:

```
df.to_csv("archivo_de_destino.csv")
```

Por otro lado, si se quiere cargar un DataFrame desde un archivo `.csv`, se utiliza el método **read_csv**:

```
df.read_csv("archivo_por_cargar.csv")
```

Notar que existen métodos similares para otras extensiones de archivo.

Scikit-learn

Adicionalmente, usaremos la librera scikit-learn, la cual provee una gran gama de técnicas de aprendizaje de máquina, tal como clasificadores del tipo **SVM**, árboles de decisión, entre otros.

Es una librería orientada a objetos, y también importaremos algunas funciones útiles. Primero que todo, para esta oportunidad nos interesan los clasificadores, que podemos importar haciendo:

```
# Clase que utilizaremos para un clasificador SVM
from sklearn.svm import SVC

# Clase que utilizaremos para un clasificador Bayesiano ingenuo
from sklearn.naive_bayes import GaussianNB

# Clase que utilizaremos para un clasificador de arbol de decision
from sklearn.tree import DecisionTreeClassifier
```

A la hora de trabajar con estas clases se debe tener en cuenta lo siguiente:

- Todos los clasificadores pueden ajustar sus hiperpárametros con el método `set_params`.
- El método `fit(X, y)` ejecutará el procedimiento de optimización de parámetros o entrenamiento del clasificador.
- El método `predict(X)` generará un vector `y` con la predicción del clasificador para la matriz `X`.

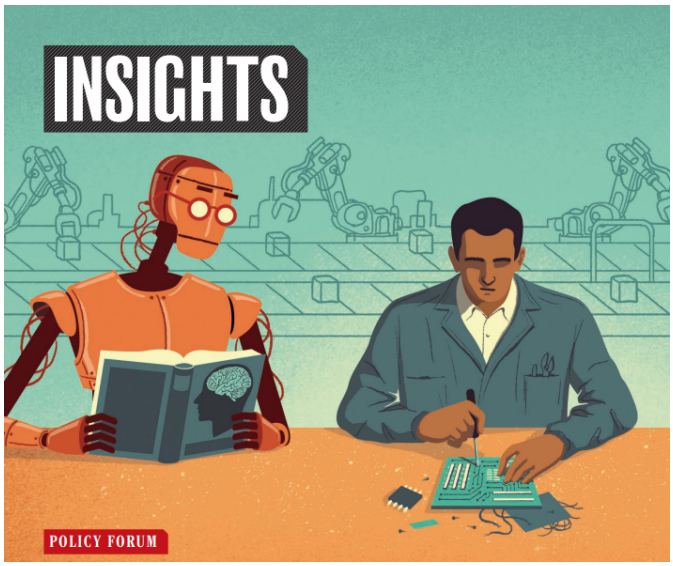
Google Colab

Si queremos usar estos algoritmos de manera más rápida, podemos usar las famosas GPUs (tarjetas gráficas). Estas tarjetas son procesadores diseñados para realizar tareas gráficas, pero también han demostrado una gran utilidad en el campo del aprendizaje automático por su alto poder de procesamiento en paralelo. Google ha dispuesto una herramienta gratuita para poder aprovechar su uso. Esta corresponde a Google Colab, por lo que recomendamos fuertemente desarrollar la tarea en esta plataforma, que además permite exportar el archivo como un archivo **.ipynb**.

IMPORTANTE: Para poder activar el uso de GPU en Google Colab, deben ir a *Editar* → *Configuración del notebook* → *Acelerador de hardware* → *GPU*.

1. Reflexión (1.5 puntos)

Para esta sección de la tarea deberás leer el artículo *What can machine learning do? Workforce implications* de Brynjolfsson y Mitchell (2017), el archivo se encuentra dentro del subdirectorio `reflexion`.



1.1 Dentro del artículo se menciona la sigla **SML** (*suitable for Machine Learning*). Este término se utiliza para describir las tareas que pueden ser realizadas, o mejoradas, por sistemas de aprendizaje automático. Los autores proponen una rúbrica de 8 criterios para distinguir si una tarea es SML o no, en base a estos criterios responde:

- ¿Cuáles de estos criterios crees que se mantienen hoy en día? ¿Cuáles crees que no se mantienen? Justifica (0.5 puntos).
- Analizando la carrera que estás estudiando actualmente ¿Cuáles de tus tareas crees que cumplen con criterios de SML? ¿Cuáles no? Justifica (0.5 puntos).

1.2 El artículo resume el efecto global del aprendizaje automático en seis factores económicos. Escoge 2 de ellos y analiza cómo podrían transformar tu futuro laboral (0.5 puntos).

¿Qué entregar en esta parte?

En un subdirectorio `reflexion/`:

- Un archivo `respuesta.pdf` con las respuestas a las preguntas.

2. Mi primera DCCasa (2.25 puntos)

2.1. Introducción

Durante el último tiempo, el mercado inmobiliario ha experimentado una fuerte transformación impulsada por la digitalización y el acceso masivo a datos. Hoy en día, existen múltiples plataformas que recopilan información detallada sobre propiedades: desde características físicas hasta información geográfica y descripciones escritas por los propios vendedores o corredoras. Sin embargo, a pesar de la gran cantidad de información disponible, determinar el precio adecuado de una propiedad sigue siendo una tarea compleja.En particular, es importante distinguir que el valor real de una propiedad, conocido como tasación, es fundamental en el sistema financiero, ya que permite a los bancos determinar si otorgan un crédito hipotecario y por qué monto, además de ser utilizado para estimar el valor de seguros asociados. El valor de una vivienda no depende únicamente de una variable aislada, sino que surge de la interacción de múltiples factores: tamaño, ubicación, antigüedad, equipamiento, entre otros.En este contexto, una conocida corredora de propiedades ha decidido dar un paso hacia la modernización de sus procesos. Para ello, ha recopilado un conjunto de datos históricos con información de propiedades publicadas, con el objetivo de construir una herramienta que permita estimar automáticamente la tasación de una propiedad en UF. Dado tu conocimiento en análisis de datos y *machine learning*, te han contactado para desarrollar un primer prototipo de este sistema. Tu misión será analizar los datos disponibles, preparar la información y entrenar distintos modelos predictivos que permitan aproximar el valor de una propiedad en base a sus características.

Este proyecto corresponde a tu primera incursión en el mundo inmobiliario... tu primera DCCasa.

2.2. Datos

Se te entregara el dataset `propiedades.csv` que contiene información sobre distintas propiedades. Cada fila representa una propiedad distinta, mientras que cada columna corresponde a una característica asociada a ella.

Las variables disponibles son las siguientes:

- **ID**: identificador único de la propiedad.
- **ID_TIPO_PROPIEDAD**: indica el tipo de propiedad (casa o departamento).
- **DESCRIPCIÓN**: descripción textual de la propiedad.
- **HABITACIONES**: número de habitaciones.
- **BANOS**: número de baños.
- **DIMENSIÓN_PROPIEDAD**: superficie construida (en m^2).
- **DIMENSIÓN_TERRENO**: superficie del terreno (en m^2).
- **DIRECCION**: dirección principal de la propiedad.
- **DIRECCION_2**: dirección secundaria de la propiedad.
- **ID_COMUNA**: identificador único de la comuna en la que se ubica la propiedad.
- **ID_PROVINCIA**: identificador único de la provincia en que se ubica la propiedad.
- **ID_REGION**: identificador único de la región en que se ubica la propiedad.
- **ID_ANTIGUEDAD**: categoría de antigüedad de la propiedad.
- **YEAR**: año en que se construyó la propiedad.
- **GASTOS_COMUNES**: gastos comunes mensuales de la propiedad.
- **PISO**: piso en que se encuentra la propiedad.
- **ESTACIONAMIENTOS**: cantidad de estacionamientos que tiene la propiedad.
- **AMOBLADO**: indicador de si la propiedad se vende amoblada o no.
- **CANTIDAD_CARACTERISTICAS**: cantidad de características adicionales con las que cuenta la propiedad.
- **PRECIO_EN_UF**: precio de la propiedad en UF.

Es importante notar que el dataset contiene variables de distinta naturaleza: numéricas, categóricas y textuales. Por lo tanto, será necesario tomar decisiones informadas respecto a su tratamiento antes de entrenar los modelos.

2.3. Actividades

A continuación se describirán las actividades que debes realizar. Recuerda que toda decisión debe ser justificada, y que cualquier resultado debe ser analizado. Si se entrega código sin explicación, este no será considerado.

2.3.1. Análisis exploratorio de Datos (0.3 puntos):

Antes de construir cualquier modelo predictivo, es fundamental comprender en profundidad la estructura y calidad del dataset. Un buen análisis exploratorio permite detectar problemas en los datos, identificar patrones relevantes y orientar decisiones posteriores de modelamiento.

En esta sección, se espera que realices un análisis exploratorio riguroso del dataset entregado.

En particular se espera:

- Analizar la distribución de la variable objetivo **`PRECIO_EN_UF`**. Comenta si presenta asimetría, valores extremos o comportamiento no normal, y cómo esto podría afectar el modelamiento.
- Identificar la presencia de valores faltantes en el dataset. Indica en qué variables aparecen, su magnitud relativa y discuta posibles estrategias para tratarlos.
- Estudiar la relación entre variables numéricas relevantes y el precio, por ejemplo:
  - **`DIMENSIÓN_TERRENO`**
  - **`DIMENSIÓN_PROPIEDAD`**
  - **`BANOS`**
  - **`HABITACIONES`**

  Puedes apoyarte en gráficos de dispersión u otras visualizaciones.
- Analizar la correlación entre variables numéricas y discutir si existen variables altamente correlacionadas entre sí.
- Detectar posibles outliers o valores inconsistentes en el dataset, explicando por qué podrían ser problemáticos.
- Identificar, de manera preliminar, qué variables podrían ser más relevantes para predecir el precio de una propiedad.

Debes complementar tu análisis con visualizaciones y estadísticas descriptivas, explicando claramente tus principales hallazgos. No basta con mostrar gráficos: se espera una interpretación de estos. **Además, algunos datos pueden no estar en el formato correcto, por lo que es importante revisarlos al momento de graficar, ya que podrían alterar los resultados.**

2.3.2. Procesamiento de datos (0.3 puntos):

Una vez comprendidos los datos, el siguiente paso consiste en prepararlos adecuadamente para el entrenamiento de modelos de *machine learning*. Las decisiones tomadas en esta etapa pueden tener un impacto significativo en el desempeño final. En esta sección, se espera que construyas un pipeline de preprocesamiento adecuado.

Debes:

- Proponer y aplicar una estrategia para el manejo de valores faltantes. Justifica tu decisión (por ejemplo, eliminación de filas, imputación con media/mediana, etc.).
- Transformar variables categóricas en un formato adecuado para los modelos. Puedes utilizar técnicas como *one-hot encoding* u otras, justificando su elección.
- Analizar el rol de las variables textuales (`DESCRIPCION`, `DIRECCION`, etc.). Puedes optar por:
  - eliminarlas

- simplificarlas,
- o aplicar algun tipo de procesamiento básico

Cualquier decisión debe estar justificada.

- Evaluar si es necesario escalar o normalizar variables numéricas, considerando los modelos que se utilizarán posteriormente.
- Eliminar variables que consideres irrelevantes, redundantes o que no aporten valor al modelo.

Se evaluará tanto la correcta aplicación de técnicas como la justificación de las decisiones tomadas.

2.3.3. División de Datos (0.15 puntos):

Antes de entrenar los modelos, es necesario dividir el dataset en subconjuntos que permitan evaluar correctamente su desempeño.
En esta sección, debes:

- Separar el dataset en conjunto de entrenamiento, conjunto de validación y conjunto de prueba.
- Indicar claramente la proporción utilizada (por ejemplo, 80 % entrenamiento, 10 % validación, 10 % prueba).
- Justificar tu elección en función del tamaño del dataset y el objetivo del problema.

ADVERTENCIA: Esta separación debe realizarse antes de aplicar cualquier transformación que pueda inducir *data leakage*. En caso contrario, los resultados obtenidos podrían ser optimistas y no reflejar el desempeño real del modelo.

2.3.4. Modelos de Regresión (0.75 puntos):

El objetivo de esta sección es entrenar distintos modelos de regresión que permitan estimar el precio de una propiedad a partir de sus características.
Debes implementar al menos los siguientes modelos:

1. Regresión Lineal
2. Árbol de Regresión
3. Un tercer modelo de regresión a elección (por ejemplo: Random Forest, Gradient Boosting, entre otros)

Para cada modelo, se espera que:

- Entrenes el modelo utilizando exclusivamente el conjunto de entrenamiento.
- Ajustes hiperparámetros si lo consideras necesario (puede justificar si decide no hacerlo).
- Expliques brevemente el funcionamiento del modelo, destacando sus principales supuestos y características.
- Comentes, de forma preliminar, qué tipo de relaciones es capaz de capturar (lineales, no lineales, interacciones, etc.).

No es necesario realizar una optimización exhaustiva, pero sí se espera una implementación correcta y razonada.

2.3.5. Evaluación y comparación de modelos (0.75 puntos):

Una vez entrenados los modelos, es necesario evaluar su desempeño utilizando el conjunto de prueba. En esta sección, debes:

- Calcular al menos dos métricas de evaluación apropiadas para regresión (por ejemplo: MAE, MSE, RMSE, R^2).
- Presentar los resultados de cada modelo de manera clara (tabla o resumen).

Luego, debes realizar un análisis comparativo que incluya:

- Identificación del modelo con mejor desempeño.
- Discusión sobre por qué algunos modelos funcionan mejor que otros en este contexto.
- Análisis de posibles problemas como overfitting o underfitting.
- Reflexión sobre cómo las características del dataset influyen en los resultados.

Se evaluará especialmente la calidad del análisis, más que solo los valores numéricos obtenidos.

¿Qué entregar en esta parte?

Para esta actividad se deberá entregar un archivo DCCasa.ipynb en el repositorio **con todas las celdas ejecutadas, de lo contrario no será corregido**. Dicho notebook deberá poder funcionar al momento de la corrección para obtener el puntaje. Aquí debes incluir tanto el código como las respuestas teóricas. Recuerda que debes justificar tus decisiones y explicar tu procedimiento, para cada una de las actividades.

Importante: No debes subir los datos entregados a tu repositorio de github, de lo contrario habrá un penalización importante en tu nota.

3. DCCoches Robados (2.25 puntos.)

3.1. Introducción

Es una mañana de lunes y, mientras revisas las noticias, notas un titular preocupante: el robo de vehículos ha aumentado significativamente en el último año. Como estudiante de Inteligencia Artificial en el DCC, no puedes evitar preguntarte: **¿Es posible predecir si un vehículo robado será recuperado basándose en las condiciones del siniestro?**

Esa misma tarde, recibes un correo de un antiguo compañero que ahora trabaja en el **SEBV (Servicio de Encargo y Búsqueda de Vehículos)**. Te comenta que tienen una base de datos histórica de siniestros, pero que el proceso de clasificación manual es lento. Necesitan una herramienta que, mediante aprendizaje automático, ayude a priorizar casos o entender qué factores (comuna, marca, año, etc.) influyen más en el éxito de una recuperación.

El objetivo de esta sección es desarrollar modelos de clasificación supervisada para predecir el estado de búsqueda de un vehículo. Utilizarás un dataset con información real de siniestros, que incluye desde datos técnicos del vehículo hasta el relato de los hechos. A diferencia de problemas de regresión, aquí te enfrentarás a un **problema de clasificación binaria**: determinar si un vehículo terminará en estado `SEBV ENCONTRO` o `SEBV BUSCA`.

3.2. Datos

Para esta actividad se entrega el archivo `datos_coches.csv`. Este dataset contiene información de 7.133 siniestros y cuenta con las siguientes columnas principales:

- **Identificadores:** `id_prose`, `sin_patente`, `sin_siniestro`.
- **Información del Vehículo:** `mar_desc` (marca), `sin_ano_vechiculo` (año), `tve_desc` (tipo de vehículo), `sin_valor_comercial_veh`.
- **Ubicación y Tiempo:** `reg_description_comuna`, `pro_description_provincia`, `sin_fecha_siniestro`, `sin_hora_siniestro`.
- **Relato y Detalles:** `sin_relato` (descripción textual), `sin_dispositivos_vehiculo`.
- **Variable Objetivo:** `esp_desc` (Estado de búsqueda SEBV).

3.3. Actividades

A continuación se describirán las actividades que debes realizar. Recuerda que toda decisión debe ser justificada, y que cualquier resultado debe ser analizado. Si se entrega código sin explicación, este no será considerado.

3.3.1. Análisis exploratorio de Datos (0.3 puntos)

Para trabajar con los datos entregados se espera que primero puedan analizarlos a profundidad. Para esto deberás primeramente:

1. Cargar los datos desde el archivo **datos_coches.csv** y comentar sobre la estructura general de los datos:
   - ¿Cuántos datos hay?
   - ¿Qué atributos tiene y de qué tipo son?
   - ¿Cómo se distribuyen los datos respecto a la variable que queremos predecir?

2. Luego deberás realizar un análisis exploratorio. Para ello te pedimos que realices al menos 3 gráficos que permitan comprender mejor los datos. Por ejemplo:

   - La cantidad de datos por cada categoría de `esp_desc`.
   - La relación entre la marca del vehículo y la categoría de `esp_desc`.
   - La relación entre el año del vehículo y la categoría `esp_desc`.
   - Otras variable que consideres relevantes.

   Comenta sobre lo que observas en cada gráfico y qué información te entrega sobre el problema a resolver.

3.3.2. Procesamiento de datos (0.3 puntos)

Previo a entrenar un modelo de aprendizaje automatico, se espera que los datos estén procesados y sean adecuados para el modelo.

- Primeramente, analiza las columnas del dataset y responde ¿Son todas las columnas presentes útiles para una predicción? ¿Que atributos no aportan información? Decide que hacer con estos atributos.
- Continua identificando que transformaciones son necesarias para que los modelos puedan utilizar los datos. Ten en cuenta que estos esperan datos numéricos como atributos, pero actualmente tienes algunos categóricos o de otra naturaleza. Decide cual transformación es la mejor para cada atributo. (Utilizando One-Hot Encoding o Label Encoding por ejemplo)
- Si lo estimas adecuado, puedes generar columnas adicionales (booleanos, número de veces que aparece un cierto dato en la columna, etc.)

3.3.3. División de Datos (0.15 puntos)

Antes de realizar cualquier entrenamiento y con el fin de posteriormente poder evaluar los modelos se deberán dividir los datos de la siguiente forma.

- Divide el dataset en un conjunto de entrenamiento, validación y prueba. Para esto, deberás decidir que proporción de los datos corresponderá a cada categoría, justifica tu decisión.
- Separa el dataset en variables independientes y variables dependientes (esp_desc)

3.3.4. Entrenamiento de modelos (0.75 puntos)

A continuación deberás entrenar modelos predictivos de aprendizaje automático para resolver la tarea de predecir el valor de esp_desc. Específicamente deberás entrar los modelos de **Decision Tree, Random Forest y SVM** siguiendo el siguiente procedimiento para cada uno.

- Identifica qué hiperparámetros son los más importantes y explica que efectos tienen sobre el modelo.
- Busca la mejor combinación de hiperparámetros en el modelo. Para esto implementa una estrategia de validación. Se recomienda utilizar Grid Search o Random Search. Justifica tu elección y evalua algunas combinaciones de hiperparámetros para seleccionar la mejor dentro de las combinaciones probadas.
- Finalmente selecciona los mejores hiperparámetros y documenta sus resultados.

Advertencia: Ten en cuenta que entrenar un modelo es una tarea computacionalmente exigente. Así que es posible que esta parte tome unos minutos. Para limitar el tiempo, debes limitar también la cantidad de parámetros que se prueban.

3.3.5. Análisis de resultados (0.75 puntos)

Una vez entrenados y optimizados los modelos, es necesario evaluar su capacidad de generalización y entender la lógica detrás de sus predicciones. Para esta sección, realiza lo siguiente:

1. **Evaluación de Métricas:** Evalúa los modelos finales en el conjunto de prueba utilizando: *Accuracy*, *Precision*, *Recall* y *F1-Score*. Explica brevemente qué indica cada una en el contexto de la recuperación de vehículos por el SEBV.

2. **Reflexión Teórica (Ensemble Learning):** Explica conceptualmente por qué **Random Forest** se define como un método de **ensamble**. Discute cómo el uso de múltiples árboles de decisión y la técnica de promediar respuesta ayudan a mejorar la robustez del modelo frente al sobreajuste (*overfitting*) en comparación con un solo árbol de decisión.

3. **Tabla Comparativa y Diagnóstico:** Presenta una tabla comparativa con todas las métricas calculadas para los conjuntos de **entrenamiento** y **prueba**. ¿Observas señales de *overfitting* o *underfitting*? Basado en esto, justifica cuál modelo es el mejor para este problema.

4. **Visualización del Desempeño:** Crea al menos dos visualizaciones. Se sugiere la **Matriz de Confusión** y un gráfico de **Importancia de Atributos** (Feature Importance) para el modelo Random Forest. Comenta qué variables resultaron ser las más influyentes para predecir si un auto será encontrado.

5. **Análisis Crítico de Errores:** Identifica en qué escenarios tu mejor modelo falla más (ej. ciertas comunas, marcas o tipos de vehículos). Realiza un análisis crítico sobre las limitaciones técnicas y éticas de tu modelo en un contexto de seguridad pública.

¿Qué entregar en esta parte?

Para esta actividad se deberá entregar un archivo DCCoches.ipynb en el repositorio **con todas las celdas ejecutadas, de lo contrario no será corregido**. Dicho notebook deberá poder funcionar al momento de la corrección para obtener el puntaje. Aquí debes incluir tanto el código como las respuestas teóricas. Recuerda que debes justificar tus decisiones y explicar tu procedimiento, para cada una de las actividades.

Importante: No debes subir los datos entregados a tu repositorio de github, de lo contrario habrá un penalización importante en tu nota.